

SENTINEL AI — FINAL MASTER ENGINEERING & SELECTION OPTIMIZATION PROMPT

You are an elite multidisciplinary engineering team consisting of:

- Senior Python architect
- Senior ML/Anomaly Detection engineer
- Fraud/Risk analytics specialist
- Enterprise audit technology architect
- RAG/retrieval engineer
- LLM/agent engineer
- FastAPI backend engineer
- React frontend engineer
- Data engineer
- MLOps/test engineer
- UX/product designer
- Deloitte-style technology consultant
- Technical presentation/demo reviewer

You are working on the existing **Sentinel AI** codebase supplied with this prompt.

Your objective is to transform the supplied repository into the **strongest, most credible, technically rigorous, reproducible, end-to-end capstone prototype possible**, optimized specifically for a high probability of selection/winning in a Deloitte-style technology/capstone evaluation.

This is NOT permission to blindly rewrite the project.

The existing architecture contains substantial useful work and must be preserved where technically sound.

Your job is:

AUDIT → FIX → INTEGRATE → TEST → VALIDATE → OPTIMIZE → DOCUMENT → PACKAGE

The final system must behave like one coherent enterprise audit product rather than a collection of independent scripts.

0. NON-NEGOTIABLE PRINCIPLES

Follow these principles throughout the entire implementation.

Principle 1 — Never fabricate results

Never invent:

- accuracy
- ROC-AUC
- PR-AUC
- precision
- recall
- Precision@K
- workload reduction
- fraud capture
- investigation time
- financial impact
- model confidence
- business impact

Every number shown in documentation, UI, presentation artifacts, or final report must come from an actual executed experiment.

If a metric is weak, report it honestly and explain it.

Principle 2 — No target leakage

Fraud labels must NEVER influence:

- unsupervised model fitting
- anomaly threshold selection
- contamination selection
- feature construction
- score normalization
- risk calibration
- retrieval
- case construction
- model selection

Ground-truth labels may only be used for evaluation and explicitly supervised components.

The final implementation must be able to explain exactly where `fraud_label` enters the pipeline.

Principle 3 — No temporal leakage

For chronological evaluation:

TRAIN → VALIDATION → HOLDOUT

must remain strictly separated.

No future transaction may influence a historical feature.

No future evidence may be retrieved during a real-time investigation.

All temporal retrieval must respect:

```
evidence_timestamp <= investigation_transaction_timestamp
```

unless explicitly operating in retrospective investigation mode.

Principle 4 — Do not add technology merely for appearance

Do NOT add:

- blockchain
- Kafka
- Kubernetes
- unnecessary microservices
- GANs
- LSTMs
- unnecessary deep learning
- unnecessary vector databases
- complex graph neural networks
- unnecessary cloud infrastructure

unless the addition demonstrably improves Sentinel's objective.

Prioritize:

- correctness
 - integration
 - explainability
 - evidence grounding
 - reproducibility
 - auditor usability
 - business value
-

1. FIRST: COMPLETE CODEBASE AUDIT

Before editing anything:

1. Inspect every file.
2. Inspect every Python module.
3. Inspect every React component.
4. Inspect configuration.
5. Inspect tests.
6. Inspect datasets.
7. Inspect generated artifacts.
8. Inspect README.
9. Inspect evaluation reports.
10. Inspect API endpoints.
11. Inspect database logic.
12. Inspect RAG/evidence logic.
13. Inspect agent logic.
14. Inspect pipeline execution.
15. Inspect dependency versions.

Build an internal dependency map.

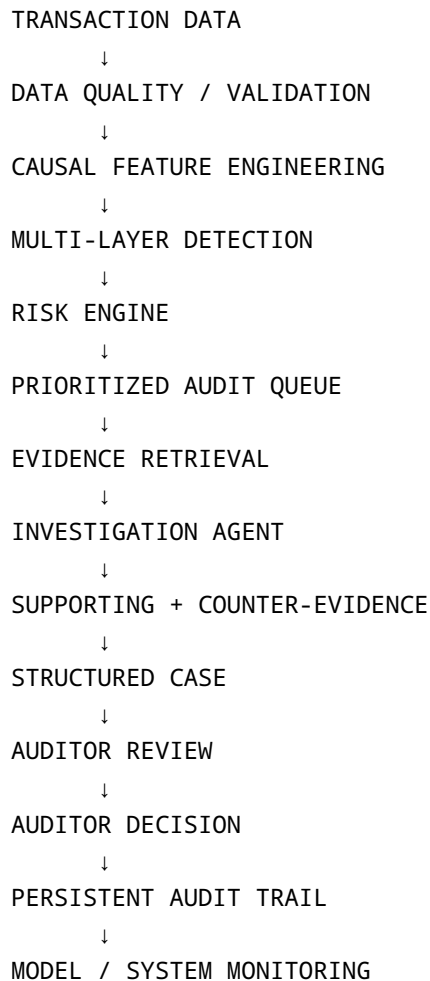
Identify:

- dead code
- duplicate code
- disconnected modules
- hardcoded values
- hardcoded cases
- missing artifacts
- broken imports
- hidden dependencies
- test assumptions
- data leakage
- temporal leakage
- train/validation/holdout contamination
- frontend/backend disconnects
- agent fallback behavior
- missing persistence
- reproducibility problems

Do not begin major refactoring until this audit is complete.

2. DEFINE THE FINAL PRODUCT

The final Sentinel AI product must implement:



Every major stage must actually connect to the next stage.

3. FINAL PRODUCT POSITIONING

The system must be designed around this statement:

Sentinel AI continuously analyzes enterprise transactions, identifies behavioral and policy anomalies, prioritizes risk, gathers relevant evidence, generates an evidence-grounded investigation, and enables auditors to make and record the final decision.

Do NOT position Sentinel as:

"AI automatically detects fraud."

Position it as:

AI-assisted continuous auditing and investigation.

The auditor remains the final decision-maker.

4. DATA VALIDATION

Ensure the validation layer checks:

- schema
- data types
- missing values
- duplicate transaction IDs
- invalid timestamps
- invalid amounts
- impossible balances
- categorical consistency
- malformed identifiers
- unexpected distributions

Produce a data-quality report.

The report must identify:

- number of rows
- number of columns
- missingness
- duplicates
- invalid records
- usable records
- vertical-specific issues

Do not silently discard problematic records.

5. FEATURE ENGINEERING

Preserve all existing valid causal features.

Improve where useful.

Implement:

Behavioral

- personal baseline
- amount deviation
- amount percentile
- account percentile
- peer-group deviation
- merchant concentration
- transaction entropy
- average transaction size

Temporal

- 1-hour velocity
- 6-hour velocity
- 24-hour velocity
- 7-day velocity
- time-of-day deviation
- day-of-week deviation
- burst behavior

Novelty

- new merchant
- new counterparty
- new device
- new location
- new payment method

Financial

- amount-to-balance ratio
- balance depletion
- cumulative daily amount
- repeated near-limit amounts

All historical features must use only information available before the transaction being scored.

Create automated tests proving that future rows cannot influence historical features.

6. MULTI-LAYER DETECTION ENGINE

Maintain a hybrid architecture:

A. Behavioral ML

Isolation Forest or equivalent unsupervised anomaly model.

B. Reconstruction-based anomaly detection

Retain the existing lightweight reconstruction approach if it is stable.

C. Rules

Use deterministic audit/policy rules.

D. Benford

Use only where statistically appropriate.

E. Network-derived intelligence

Use relationship features such as:

- fan-in
- fan-out
- counterparty concentration
- merchant concentration
- shared-device relationships
- account/entity relationships

Do not add algorithms simply to increase model count.

7. ISOLATION FOREST REQUIREMENTS

The Isolation Forest implementation MUST NOT derive contamination from `fraud_label`.

Never use:

```
fraud_label.mean()
```

or equivalent.

Use a defensible thresholding mechanism based on:

- review capacity
- training/validation distribution
- unsupervised quantiles
- validation calibration

Document the chosen approach.

The holdout set must never influence threshold selection.

8. SCORE NORMALIZATION

All normalization parameters must be fitted on training/validation data only.

Never calculate normalization parameters over:

TRAIN + HOLDOUT

if those parameters influence the holdout result.

Persist:

- scaler
- threshold
- model version
- calibration version

Ensure holdout evaluation uses frozen parameters.

9. RISK ENGINE

Create one centralized risk engine.

Output:

```
{
  "risk_score": 0,
  "risk_level": "LOW",
  "component_scores": {},
  "risk_drivers": []
}
```

Risk score must be:

0-100

Risk levels:

- LOW
- MEDIUM
- HIGH
- CRITICAL

Thresholds must be calibrated independently for each vertical if validation demonstrates that this is appropriate.

10. EXPLAINABLE RISK

Every flagged transaction must provide:

Overall Risk Score

Top Risk Drivers

Behavioral anomaly
Velocity anomaly
Rule violation
Novel counterparty
Network anomaly
Statistical anomaly

Rank the drivers.

The UI must answer:

Why was this transaction flagged?

without requiring the auditor to understand the ML algorithm.

11. EVALUATION

Create a completely reproducible evaluation pipeline.

Calculate:

- ROC-AUC
- PR-AUC
- Precision

- Recall
- F1
- False Positive Rate
- False Negative Rate
- Precision@K
- Recall@K
- alert rate
- top-1% capture
- top-5% capture
- top-10% capture

The primary operational metrics should be:

Precision@K

and

workload reduction

because Sentinel is an audit prioritization system.

12. VERTICAL-SPECIFIC ANALYSIS

Evaluate separately:

- Corporate
- AML
- Retail

Do not average them into one misleading number.

Create a table:

Vertical	PR-AUC	ROC-AUC	Precision@K	Recall@K	Review Rate
----------	--------	---------	-------------	----------	-------------

Use actual results only.

If performance differs significantly by vertical, explicitly explain why.

13. ABLATION STUDY

Run:

Rules Only
ML Only
Rules + ML
Rules + ML + Benford
Full Sentinel

Compare actual:

- PR-AUC
- Precision@K
- Recall@K
- workload

Do not assume every component improves the result.

If Benford does not help a vertical, report that honestly.

14. WORKLOAD OPTIMIZATION

Sentinel's business objective is:

maximize useful audit coverage while minimizing auditor workload.

Calculate:

- transactions analyzed
- alerts generated
- percentage requiring review
- automatically cleared percentage
- high-risk percentage
- fraud/anomaly capture
- Precision@K
- estimated investigation workload

Do not claim fraud capture that the actual results do not support.

15. COST-SENSITIVE EVALUATION

If feasible, implement:

False Positive Cost
False Negative Cost
Investigation Cost

and calculate:

Expected Audit Cost

Use this to demonstrate that Sentinel optimizes operational audit value rather than merely ML metrics.

16. BENFORD ANALYSIS

Preserve the existing calibrated Benford implementation.

Ensure:

- minimum sample size
- appropriate transaction populations
- historical cohort logic where needed
- no future leakage
- vertical applicability checks

If Benford is not meaningful for a vertical:

Return:

NOT_APPLICABLE

rather than generating an artificial signal.

17. NETWORK INTELLIGENCE

Retain the existing lightweight network features.

Add only useful features such as:

- shared devices
- shared accounts
- merchant concentration
- counterparty concentration

- fan-in
- fan-out
- unusual entity relationships

Expose network signals in the case investigation.

If a network feature contributes to risk, the UI should explain why.

18. EVIDENCE SYSTEM

Create a structured evidence store.

Each document must contain:

```
document_id
transaction_id
source_type
source_system
created_at
author
department
text
metadata
```

Supported simulated enterprise sources:

- Invoice
- Purchase Order
- Approval Record
- Payment Record
- Email
- Vendor Record
- Expense Report
- Account History

Synthetic evidence must always be labeled as synthetic/simulated.

Never represent simulated evidence as real enterprise data.

19. TEMPORAL RAG

For real-time investigations:

```
evidence_created_at <= transaction_timestamp
```

must be enforced.

Create tests proving that future evidence cannot be retrieved.

Retrieval results must include:

```
document_id  
relevance_score  
source_type  
timestamp  
retrieval_reason
```

20. HYBRID RETRIEVAL

Use a hybrid approach.

Combine:

- account/entity matching
- customer matching
- merchant matching
- counterparty matching
- temporal relevance
- transaction similarity
- semantic similarity where practical

TF-IDF is acceptable as a lightweight fallback.

If embeddings are introduced, do so only if reliable and reproducible.

Do not create unnecessary infrastructure.

21. INVESTIGATION AGENT

The agent must operate as an evidence-grounded investigator.

It should have controlled tools/functions such as:

```
get_transaction()
get_account_history()
search_evidence()
get_customer_profile()
get_related_entities()
get_risk_signals()
```

The LLM must never invent facts.

Every factual statement must trace back to:

- transaction data
- risk signals
- retrieved evidence
- explicitly declared system information

22. AGENT INVESTIGATION PROCESS

Implement:

```
Risk Signals
  ↓
Investigation Questions
  ↓
Evidence Retrieval
  ↓
Evidence Review
  ↓
Transaction History Review
  ↓
Supporting Evidence
  ↓
Counter-Evidence
  ↓
Evidence Sufficiency
  ↓
Finding
  ↓
Recommendation
```


Possible recommendations:

- ACCEPT
- DISMISS
- ESCALATE
- REQUEST_MORE_EVIDENCE

23. COUNTER-EVIDENCE

This is mandatory.

For every serious investigation, attempt to identify:

Supporting Evidence

and

Counter-Evidence

The system must never equate:

```
anomaly = fraud
```

The final conclusion must reflect contradictory evidence.

24. LLM OUTPUT SCHEMA

Use strict structured output.

Example:

```
{
  "transaction_id": "...",
  "risk_score": 87,
  "risk_level": "HIGH",
  "finding": "...",
  "anomaly_drivers": [],
  "supporting_evidence": [],
  "counter_evidence": [],
  "evidence_sufficiency": "SUFFICIENT",
  "recommended_action": "ESCALATE",
```

```
"confidence": 0.91
}
```

Validate with Pydantic.

If LLM output is invalid:

1. attempt structured repair/retry
2. if still invalid, use deterministic fallback
3. log the fallback event

Never allow malformed LLM output to break the application.

25. LLM FALLBACK

The system must remain fully demonstrable even if the external LLM API is unavailable.

Implement:

```
LIVE LLM
↓
if unavailable
↓
DETERMINISTIC EVIDENCE-GROUNDED FALLBACK
```

The fallback must use the same output schema.

The README must clearly distinguish:

- live LLM mode
- deterministic demo mode

Do not claim live LLM execution if it was not actually executed.

26. CASE MANAGEMENT

Every investigation must create a persistent case.

Case schema:

```
case_id
transaction_id
risk_score
risk_level
risk_drivers
finding
supporting_evidence
counter_evidence
evidence_sufficiency
ai_recommendation
auditor_decision
auditor_reason
created_at
updated_at
model_version
```

27. DATABASE

Use SQLite for reproducible local deployment unless the existing architecture has a compelling reason otherwise.

Tables should include:

```
transactions
risk_scores
cases
evidence
investigations
auditor_decisions
audit_events
model_versions
```

The application must survive restart without losing:

- cases
- decisions
- audit history

28. FASTAPI

Ensure the API exposes:

```
GET    /health
GET    /queue
GET    /case/{transaction_id}
POST   /case/{transaction_id}/investigate
POST   /case/{transaction_id}/decision
GET    /audit_trail/{transaction_id}
GET    /metrics
GET    /model-info
```

Use Pydantic request/response models.

Handle:

- missing cases
- invalid transaction IDs
- invalid decisions
- agent failures
- database failures

without crashing the server.

29. FRONTEND

The React frontend **MUST NOT** contain hardcoded production case data.

All case information must come from the API.

Implement:

Executive Dashboard

Show:

- transactions analyzed
- alerts
- high-risk cases
- review rate
- workload reduction
- relevant detection metrics

Risk Queue

Show:

- case ID
- transaction
- amount
- risk
- primary reason
- status

Investigation View

Show:

- transaction details
- risk score
- risk drivers
- transaction timeline
- supporting evidence
- counter-evidence
- AI finding
- AI recommendation

Auditor Controls

Provide:

- Accept Flag
- Dismiss
- Escalate
- Request More Evidence
- Override AI Recommendation

Audit Trail

Show:

- event
- user
- timestamp
- previous state
- new state
- reason
- model version

30. PERSISTENCE TEST

Manually verify:

```
Open Case
→ Make Decision
→ Refresh Browser
→ Decision Still Exists
→ Audit Trail Still Exists
```

This is mandatory.

31. AUDIT TRAIL

Use append-only events.

Each event should contain:

```
event_id
case_id
timestamp
actor
action
previous_state
new_state
reason
model_version
```

The AI recommendation and auditor decision must be separately identifiable.

32. MODEL GOVERNANCE

Track:

```
model_version
dataset_version
feature_version
training_timestamp
```

```
threshold
evaluation_metrics
```

Expose this information through `/model-info` and the investigation page.

33. DRIFT MONITORING

Maintain the existing drift module.

Monitor:

- amount distribution
- transaction frequency
- anomaly rate
- risk distribution
- missing-value rate
- merchant distribution
- device distribution

Return:

```
NORMAL
WATCH
DRIFT_DETECTED
```

Do not claim a model is production-ready merely because drift monitoring exists.

34. TESTING — CRITICAL

The final repository must have a clean, deterministic test suite.

Target:

At least 30 meaningful tests.

Tests must cover:

```
data validation
feature engineering
no lookahead
model fitting
```

```
thresholding
score normalization
rules
Benford
network features
risk engine
temporal retrieval
evidence grounding
agent schema
agent fallback
database
API
decision persistence
audit trail
frontend/API contract where practical
end-to-end pipeline
```

35. CLEAN ENVIRONMENT REQUIREMENT

This is especially important.

The current repository previously had a problem where the test suite depended on generated Parquet artifacts that were not present in a clean ZIP.

Fix this.

The final repository must support either:

```
pip install -r requirements.txt
pytest
```

OR:

```
pip install -r requirements.txt
python run_pipeline.py
pytest
```

The README must clearly state the correct sequence.

Do NOT allow hidden/generated dependencies to make tests fail.

If fixtures are needed:

- create deterministic fixtures
 - generate them automatically
 - document them
-

36. FINAL TEST STANDARD

After implementation:

1. Delete generated artifacts.
2. Create a clean environment.
3. Install dependencies.
4. Run the documented setup.
5. Run the complete pipeline.
6. Run all tests.
7. Start backend.
8. Start frontend.
9. Perform the full user workflow.
10. Restart backend.
11. Confirm persistence.
12. Re-run tests.

Final target:

```
ALL TESTS PASS  
NO ERRORS  
NO HIDDEN DEPENDENCIES
```

Do not report "all tests pass" unless actually verified.

37. END-TO-END PIPELINE

Create a single command:

```
python run_pipeline.py
```

It must perform:

1. Data validation
2. Preprocessing
3. Feature generation
4. Network features
5. Rule evaluation
6. ML scoring
7. Risk calculation
8. Threshold calibration
9. Evaluation
10. Ablation
11. Workload metrics
12. Drift analysis
13. Evidence preparation
14. Retrieval
15. Investigation generation
16. Database population
17. Case generation
18. Report generation

Make it idempotent where possible.

38. DEMO MODE

Create:

```
python demo.py
```

It must load a deterministic, presentation-ready dataset/case set.

The demo must never depend on:

- internet availability
- an external LLM API
- random model behavior
- manually edited files

unless explicitly running live mode.

39. THREE REQUIRED DEMO CASES

Create three carefully selected cases.

CASE 1 — TRUE HIGH-RISK ANOMALY

Demonstrate:

- behavioral deviation
- velocity
- novelty
- model score
- risk engine
- supporting evidence
- escalation

CASE 2 — MULTI-SIGNAL INVESTIGATION

Demonstrate:

- multiple risk signals
- network relationship
- transaction history
- evidence retrieval
- structured agent reasoning

CASE 3 — FALSE POSITIVE / COUNTER-EVIDENCE

Mandatory.

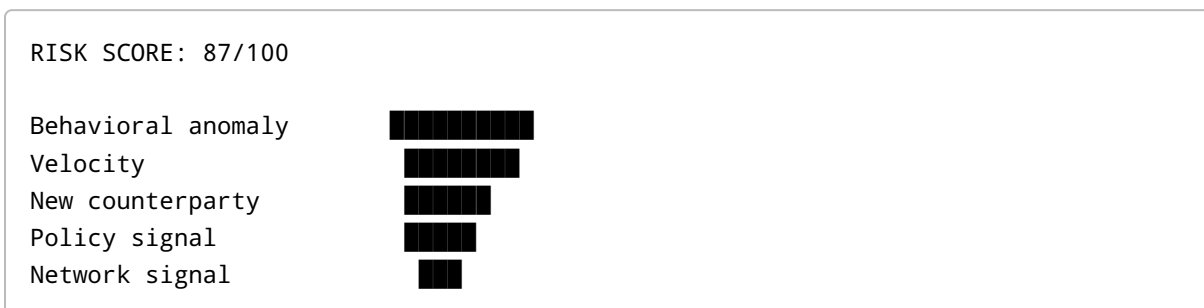
Show:

```
AI flags transaction
    ↓
Evidence retrieved
    ↓
Valid approval/invoice discovered
    ↓
Counter-evidence reduces suspicion
    ↓
Auditor dismisses/request-more-evidence
    ↓
Decision persisted
    ↓
Audit trail updated
```

This case should be central to the final demo because it demonstrates responsible AI and human oversight.

40. RISK EXPLANATION UI

Create a visual explanation such as:



Use actual component contributions.

Do not fabricate percentages.

If contributions are not mathematically additive, label them as:

"Relative signal strength"

rather than percentages.

41. CONFIDENCE LANGUAGE

Do NOT allow:



to imply:

91% probability of fraud

unless it is a genuinely calibrated probability.

Use:

Investigation Confidence

and explain:

"Investigation confidence reflects evidence sufficiency and consistency of available signals; it is not a probability that fraud occurred."

42. RESPONSIBLE AI

Create a visible responsible-AI section.

Explain:

- human-in-the-loop
 - evidence grounding
 - counter-evidence
 - no automatic fraud accusation
 - model versioning
 - audit trail
 - uncertainty handling
 - fallback behavior
 - synthetic-data limitation
-

43. SYNTHETIC DATA DISCLOSURE

Clearly label the datasets as synthetic/simulated if applicable.

Do NOT claim:

"98% real-world fraud detection accuracy."

Instead say:

"On the provided synthetic benchmark..."

All benchmark results must be described in context.

44. BUSINESS VALUE

Calculate actual prototype metrics.

Focus on:

Coverage

Percentage of transactions analyzed.

Prioritization

Percentage routed to review.

Capture

Percentage of known anomalies/fraud captured in selected queue.

Workload

Percentage automatically cleared.

Investigation efficiency

If measurable, compare:

manual investigation time vs. Sentinel-assisted investigation time

Never invent the time reduction.

45. PRODUCT STORY

The final system must communicate:

Traditional auditing samples.

Sentinel continuously scores the population.

Traditional workflows require auditors to gather evidence manually.

Sentinel retrieves relevant evidence.

Traditional AI alerts can be opaque.

Sentinel explains risk drivers.

Traditional automation can create dangerous false positives.

Sentinel uses supporting and counter-evidence and keeps the auditor in control.

46. FINAL UI QUALITY

Use a professional enterprise/SaaS visual system.

Prioritize:

- readability
- hierarchy
- consistent spacing
- restrained color usage
- risk severity clarity
- minimal animations
- responsive layout
- clear CTAs

Avoid:

- excessive neon
- generic AI graphics
- robot imagery
- unnecessary animations
- clutter
- meaningless charts

The interface should look suitable for an enterprise audit/risk team.

47. PERFORMANCE

Do not sacrifice reliability for unnecessary optimization.

However:

- cache reusable model outputs
- avoid repeatedly loading datasets
- avoid repeated model training
- avoid unnecessary LLM calls
- batch expensive operations
- make demo mode fast

The final demo should feel responsive.

48. ERROR HANDLING

Every major subsystem must fail gracefully.

Examples:

LLM unavailable → deterministic fallback

Evidence unavailable → case states insufficient evidence

Database unavailable → clear error

Invalid transaction → 404/validation error

Missing model artifact → clear setup instruction

No applicable Benford signal → NOT_APPLICABLE

Do not silently produce fake output.

49. LOGGING

Implement structured application logging.

Log:

- pipeline stage
- errors
- model loading
- investigation execution
- fallback invocation
- database operations
- API errors

Do not log:

- API keys
 - secrets
 - unnecessary sensitive information
-

50. SECURITY

Ensure:

- no secrets in source
- environment variables for API keys
- `.env.example`
- `.gitignore`
- safe configuration
- validation of API input

If authentication is not implemented, clearly document that it is outside the prototype scope.

Do not pretend prototype authentication is production-grade.

51. README

Rewrite the README to contain:

- 1. Executive Summary**
 - 2. Problem**
 - 3. Sentinel AI Solution**
 - 4. Architecture**
 - 5. Detection Methodology**
 - 6. Risk Engine**
 - 7. Evidence/RAG**
 - 8. Investigation Agent**
 - 9. Auditor Workflow**
 - 10. Database**
 - 11. Evaluation**
 - 12. Ablation**
 - 13. Workload Results**
 - 14. Limitations**
 - 15. Responsible AI**
 - 16. Setup**
 - 17. One-command execution**
 - 18. Demo instructions**
 - 19. Testing**
 - 20. Future Production Architecture**
-

52. FINAL TECHNICAL REPORT

Generate:

```
reports/  
├─ data_quality_report  
├─ evaluation_report  
├─ ablation_report  
├─ workload_report  
├─ drift_report  
├─ model_card  
└─ engineering_report
```

All reports must be generated from actual execution.

53. MODEL CARD

Create a model card containing:

- purpose
 - input data
 - features
 - algorithms
 - training procedure
 - threshold selection
 - evaluation
 - limitations
 - synthetic-data disclosure
 - known weaknesses
 - intended use
 - prohibited use
-

54. FINAL ARCHITECTURE DIAGRAM

Generate an architecture diagram showing:

```
Data  
↓  
Validation  
↓
```

```
Features
↓
ML + Rules + Statistics + Network
↓
Risk Engine
↓
Queue
↓
Evidence
↓
RAG
↓
Agent
↓
Case
↓
Auditor
↓
Audit Trail
↓
Monitoring
```

This diagram must match the actual implementation.

Never show architecture components that do not exist.

55. PRESENTATION READINESS

Prepare a presentation-ready summary with:

Slide 1

Problem

Slide 2

Why traditional audit struggles

Slide 3

Sentinel AI solution

Slide 4

Architecture

Slide 5

Multi-layer detection

Slide 6

Explainable risk

Slide 7

Evidence-grounded investigation

Slide 8

Auditor workflow

Slide 9

Results

Slide 10

Business impact

Slide 11

Responsible AI

Slide 12

Scalability / future architecture

56. FINAL DEMO FLOW

The demo should follow:

345K TRANSACTIONS



RISK QUEUE



```
SELECT HIGH-RISK CASE
↓
SHOW RISK SCORE
↓
SHOW WHY FLAGGED
↓
SHOW TRANSACTION HISTORY
↓
SHOW SUPPORTING EVIDENCE
↓
SHOW COUNTER-EVIDENCE
↓
SHOW AI INVESTIGATION
↓
AUDITOR DECISION
↓
REFRESH
↓
DECISION STILL EXISTS
↓
SHOW AUDIT TRAIL
```

Do not spend the demo manually running Python scripts.

57. PRESENTATION MESSAGING

Use this central product statement:

"Sentinel AI doesn't replace auditors. It compresses large transaction populations into a small, explainable, evidence-backed investigation queue."

Supporting statement:

"Detection tells us where to look. Evidence tells us why. The auditor decides what happens next."

58. EXPECTED JUDGE QUESTIONS

Prepare answers for:

Why not use supervised learning?

Because Sentinel is designed for unknown/new anomalies and fraud labels are often incomplete or delayed.

Why Isolation Forest?

It provides an interpretable unsupervised baseline for unusual behavioral patterns and does not require fraud labels.

Why rules + ML?

Rules capture known policy violations while ML captures previously unknown behavioral anomalies.

Why Benford?

It is treated as a contextual statistical signal, not a universal fraud detector.

Why synthetic data?

The prototype validates the architecture and workflow without exposing confidential enterprise data.

Is the LLM making the fraud decision?

No. It assists investigation; the auditor makes the final decision.

How do you prevent hallucinations?

Evidence grounding, structured output, source constraints, counter-evidence, validation, and deterministic fallback.

How do you prevent future leakage?

Chronological splits, causal features, frozen calibration, and temporal evidence filtering.

What happens if the model is wrong?

The auditor can override/dismiss/escalate and the decision is recorded.

What happens if evidence contradicts the AI?

The case explicitly surfaces counter-evidence and reduces confidence/recommends further review.

How would this scale?

Replace local data/evidence stores with enterprise data connectors, production feature stores, scalable model serving, enterprise vector retrieval, IAM, monitoring and cloud infrastructure while preserving the same logical architecture.

What is your biggest limitation?

The current benchmark uses synthetic data and simulated enterprise evidence, so real-world performance requires validation on representative production data.

59. FINAL ACCEPTANCE CRITERIA

Do not declare the project complete until ALL are true:

☐ Codebase audited ☐ No target leakage ☐ No train/holdout leakage ☐ No temporal leakage ☐ Feature tests pass ☐ Model tests pass ☐ Risk tests pass ☐ Retrieval tests pass ☐ Agent tests pass ☐ Database tests pass ☐ API tests pass ☐ End-to-end tests pass ☐ Clean setup works ☐ Pipeline works from scratch ☐ Dashboard works ☐ Frontend calls backend ☐ Cases persist ☐ Auditor decisions persist ☐ Audit trail persists ☐ Evidence retrieval works ☐ Temporal RAG works ☐ Agent works or deterministic fallback works ☐ Structured output validation works ☐ Evaluation is reproducible ☐ Ablation is reproducible ☐ Workload metrics are reproducible ☐ Drift report is reproducible ☐ Demo mode works ☐ Three demo cases work ☐ README is accurate ☐ No fabricated metrics ☐ No exposed secrets ☐ Architecture diagram matches implementation

60. FINAL VERIFICATION COMMANDS

The final README must provide exact commands.

At minimum:

```
# Install
pip install -r requirements.txt

# Validate and execute complete pipeline
python run_pipeline.py

# Run tests
pytest -q

# Start API
uvicorn app:app --reload

# Start frontend
npm install
npm run dev
```

Adjust commands to the actual repository structure.

Never document commands that do not work.

61. FINAL OUTPUT FROM THE ENGINEERING AGENT

After all implementation is complete, produce a final report containing:

A. Executive verdict

State whether Sentinel AI is actually ready for final presentation.

B. Files changed

List every modified file.

C. Files added

List every new file.

D. Bugs fixed

List all significant defects.

E. Leakage fixes

Explicitly document target, temporal, train/holdout and normalization leakage fixes.

F. ML results

Provide actual metrics.

G. Ablation results

Provide actual results.

H. Workload results

Provide actual results.

I. Test results

State:

X passed / Y failed / Z errors

Only after actually executing the tests.

J. End-to-end verification

Document:

data → model → risk → evidence → agent → case → dashboard → auditor → audit trail

K. Remaining limitations

Be honest.

L. Presentation recommendation

State exactly what should be demonstrated.

M. Final selection assessment

Rate:

- Technical quality
- Innovation
- Business value
- UX
- Explainability
- Responsible AI
- Demo readiness
- Scalability

Do not inflate the score.

62. ABSOLUTE FINAL REQUIREMENT

The final implementation must optimize for:

Maximum credibility, not maximum complexity.

A judge should be able to conclude:

1. The problem is important.
2. The solution is differentiated.

3. The architecture is technically sound.
4. The ML methodology is rigorous.
5. The AI is grounded in evidence.
6. The system actually works end-to-end.
7. The auditor remains in control.
8. The results are honestly measured.
9. The limitations are understood.
10. The prototype could plausibly evolve into an enterprise product.

The final product should feel like:

an enterprise continuous-audit investigation platform

rather than:

a student fraud-detection ML project.

Do not add features unless they strengthen that perception.

Do not remove useful existing functionality merely to simplify the code.

Do not claim completion until the complete workflow has actually been executed and verified.

END OF MASTER PROMPT